

Image Analysis and Computer Vision
Homework 2024-2025

Iusupov Safuan 11058805 - safuan.iusupov@mail.polimi.it

January 18, 2025
Prof. Vincenzo Caglioti, Politecnico di Milano.

Part 1. Feature extraction

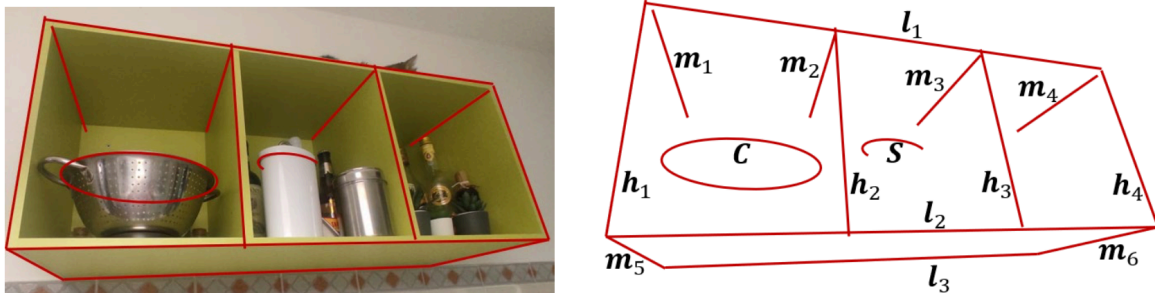


Figure 1 - Source image of furniture with lines.

For feature extraction from images, I utilized the Canny edge detection method [1], which effectively identifies both strong and weak edges in an image. Result of edge detection represented on Figure 2.



Figure 2 - Result of edge detection.

To extract lines from images, I employed the Hough Transform algorithm [2]. This method allowed me to identify and extract lines from the detected features. Result of line extraction with Hough Transform represented on Figure 3.

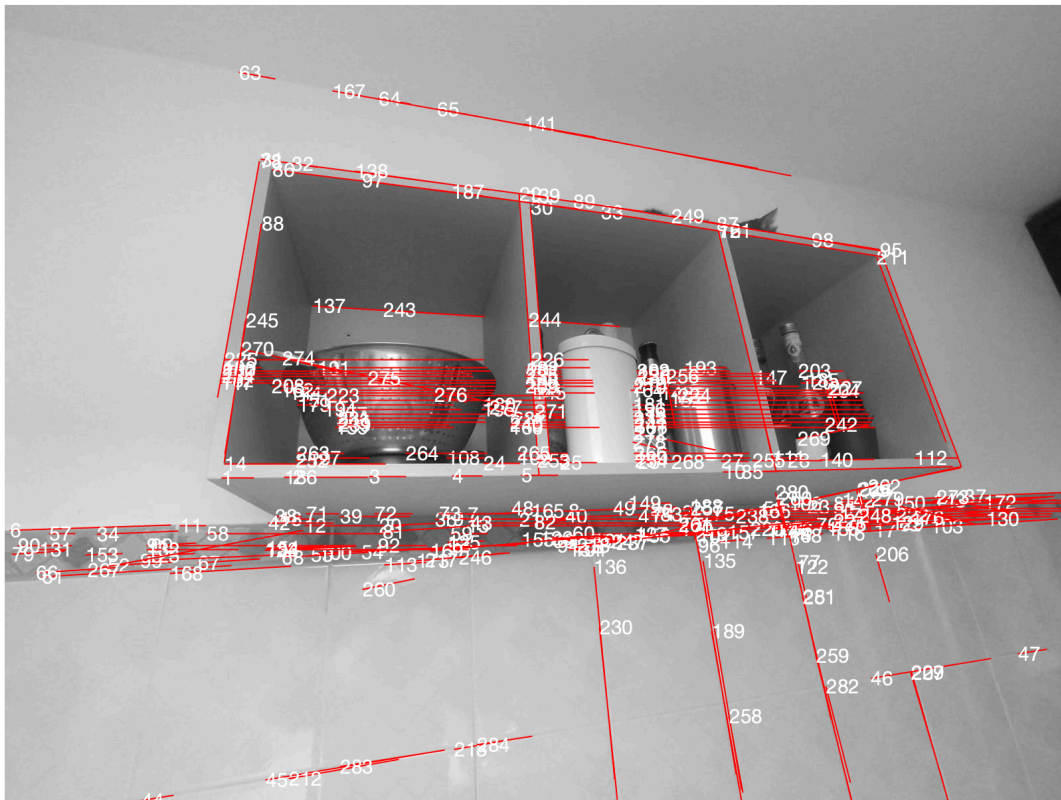


Figure 3 - Result of line detection.

To extract corners from the image, I used the Harris feature detector [3]. The results of corner detection are shown in Figure 4.

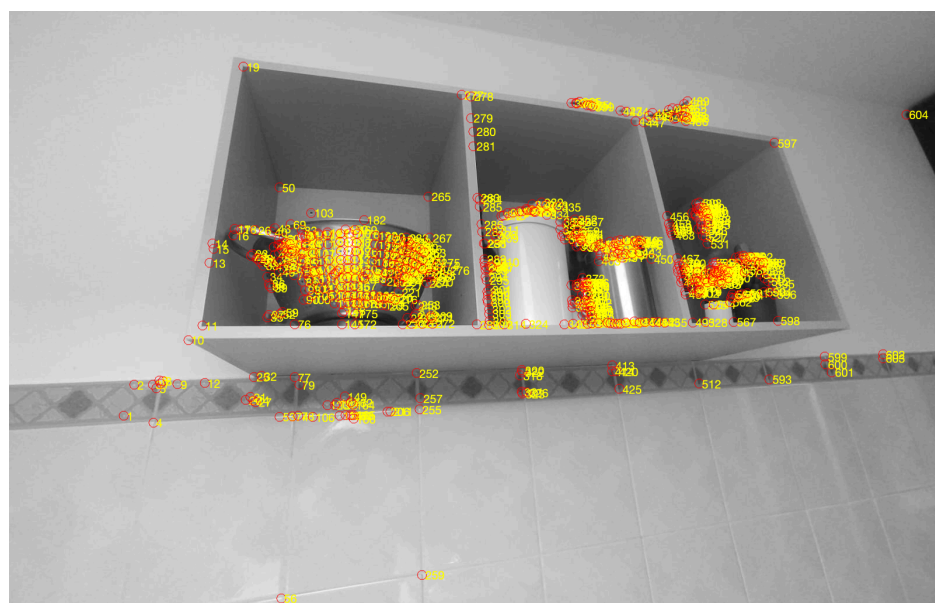


Figure 4 - Result of corners detection.

As shown in Figure 2, several lines from Figure 1, such as $m1$ and $m2$, were not successfully extracted after applying the Canny algorithm. Consequently, these lines could not be detected using the Hough Transform algorithm. The primary challenge appears to be the colander, which concentrates a significant number of edges and obscures lines in the shadowed areas ($m1$ and $m2$). Despite extensive tuning of the hyperparameters, no significant improvement in line detection was achieved. However, corner detection effectively identified the critical corners, enabling the extraction of the main lines, which will be utilized in future work. Corners that will be used in future work represented in Figure 5.



Figure 5 - Useful corners.

Vanishing line of horizontal plane

The vanishing line, also known as the horizon line, represents the image of the set of points at infinity. It can be determined using two vanishing points, which are the images of points at infinity where parallel lines in the scene converge in the image. In this case, I chose the vanishing points as the intersections of parallel image lines l_{AB} with l_{DC} and l_{AD} with l_{BC} . Result of the found vanishing line presented in Figure 6 (dashed line).

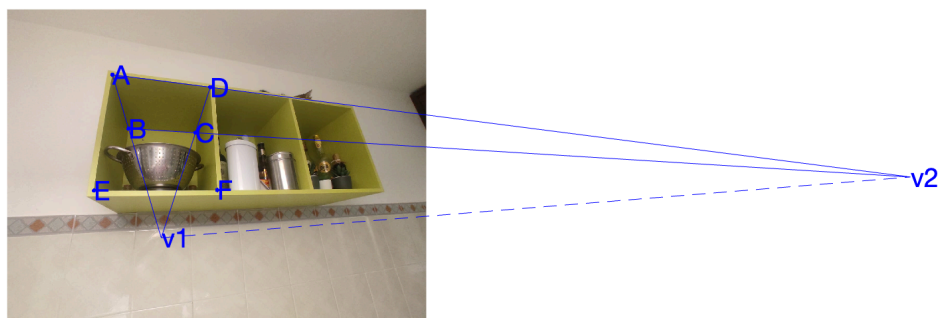


Figure 6 - Vanishing line of horizontal plane.

The value of founded vanishing line (normalised) is $[-8.815e - 05, -0.0011, 1]$. I also have tried to define vanishing point v_2 from the bottom surface of the shelf, but the lines on this surface are too close to each other and that leads to the uncertainty in the intersection point.

Part 2. Euclidean rectification mapping of horizon

Direct Method Using Singular Value Decomposition (SVD)

My initial approach to rectification involved using a direct method based on Singular Value Decomposition (SVD). To achieve this, I first defined the image of the conic dual to the circular points, denoted as C_∞^* . This process required determining the image of the circular points I' and J' . By definition, circular points are the intersections of the circumference and the line at infinity. Given the vanishing line l_∞ (computed from the previous step), I formulated a system of equations to determine the coordinates of I' and J' . With I' and J' known, I was able to compute C_∞^* .

$$\begin{cases} ax^2 + bxy + cy^2 + dx + ey + f = 0 \\ v_x x + v_y y + v_w = 0 \end{cases}$$

where a, b, c, d, e, f – coefficients of image conic
 v_x, v_y, v_w – coefficients of vanishing line

Next, I performed SVD on C_∞^* to decompose it into its eigenvalues and eigenvectors. However, the resulting C_∞^* contained negative eigenvalues, which rendered this method unsuitable for rectification.

$$C_\infty^* = I' J'^T + J' I'^T$$

$$SVD(C_\infty^*) = U \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & 0 \end{bmatrix} U^T = H_{SR}^{-1} C_\infty^* H_{SR}^{-T}$$

$$H_{SR} = \begin{bmatrix} 1/\sqrt{(a)} & 0 & 0 \\ 0 & 1/\sqrt{(b)} & 0 \\ 0 & 0 & 1 \end{bmatrix} U^T$$

Therefore I used **Stratified Rectification**, which contain two main parts:

1. **Affine reconstruction** - from projective to affine.

Using the result of the previous point I could define a Homography that maps the image of the line at infinity to its canonical position. Affine reconstruction can be found as:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ v_x & v_y & v_w = 1 \end{bmatrix}$$

where v_x, v_y, v_w — components of vanishing line (image of infinity line)

$$H_{aff} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -8.815e-05 & -0.0011 & 1 \end{bmatrix}$$

Result of Affine rectification presented in Figure 7.

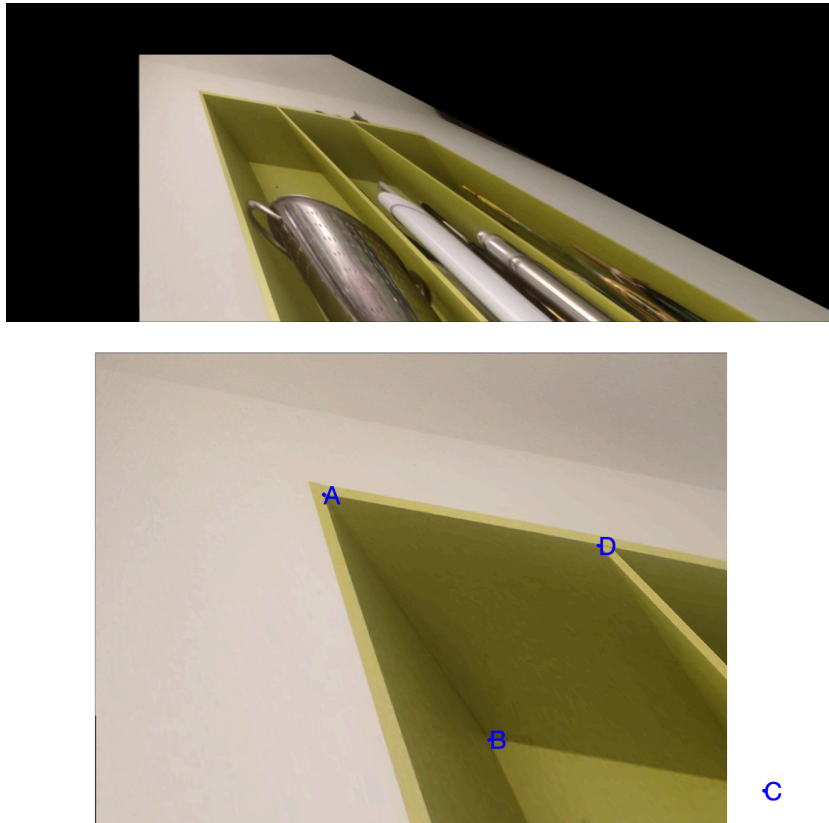


Figure 7 - Affine image rectification (top - rectification of full img, bottom - rectification of local region).

To verify the correctness of rectification I computed the angles between AB with DC and BC with AD (they have to be parallel), which was equal to zero, that proves my transformation. I also verified it by transforming the vanishing line with received transform, the result has to be a canonical form of infinity line.

$$l_{\infty} = H_{aff}^{-T} v = [0, 0, 1]^T$$

2. Metric Rectification: From Affine to Metric

After achieving affine rectification, the next step was to perform metric rectification. The goal was to restore the true metric properties of the scene, such as angles and aspect ratios. I made a transformation, which converts the ellipse to the circle (axis has to be equal and orthogonal).

$$H_{metric} = \begin{bmatrix} K & 0 \\ 0 & 1 \end{bmatrix}$$

$$K = USU^T$$

$$S = \begin{bmatrix} 1 & 0 \\ 0 & a/b \end{bmatrix} - \text{scaling matrix (to make axis equal)}$$

where a, b - lengths of ellipse axis

$$U = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix} - \text{rotation matrix (to make axis orthogonal)}$$

α, a, b - can be found from geometric parameters of ellipse C_{aff} , after applying H_{aff} :

$$C_{aff} = H^{-T}CH^{-1}$$

$$H_{metric} = \begin{bmatrix} 1.4888 & -0.7169 & 0 \\ -0.7169 & 2.0515 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\text{Resulting transform } T = H_{metric} \cdot H_{aff} = \begin{bmatrix} 1.4888 & -0.7169 & 0 \\ -0.7169 & 2.0515 & 0 \\ -0.0001 & -0.0011 & 1.0000 \end{bmatrix}$$

To apply this transform I transposed the matrix. Base ellipse C used in this exercise presented in Figure 8, final transformation is composition of affine and metric rectification, results presented in Figure 9.



Figure 8 - ellipse *C*.



B

C

Figure 9 - Result of Euclidean rectification mapping (top - rectification of full img, bottom - rectification of local region) of horizontal plane.

To verify my rectification I computed the angles between AB with AD and DC with BC , which was close to 90 degrees.

Given the total length of the parallelepiped and the information that AD is one-third of that length, we can determine the length of AB . By using this proportion, the calculated length of AB is **0.33957**. Most of the functions and ideas used in this part were taken from **Luca Magri practice sessions** [4].

Part 3. Calibration Matrix

Our camera is **zero-skew**, which means that the calibration matrix has four independent variables. So we need to use four constraints to define them. I used relationships between w IAC (Image of Absolute Conic) and K (calibration matrix): $w = (KK^T)^{-1}$, where w is a symmetric matrix and K is:

$$K = \begin{bmatrix} f_x & 0 & u_x \\ 0 & f_y & u_y \\ 0 & 0 & 1 \end{bmatrix}, w = \begin{bmatrix} 1/f_x^2 & 0 & -u_x/f_x^2 \\ 0 & 1/f_y^2 & -u_y/f_y^2 \\ -u_x/f_x^2 & -u_y/f_y^2 & 1 + u_x^2/f_x^2 + u_y^2/f_y^2 \end{bmatrix}$$

IAC in terms of calibration matrix parameters I found using Matlab symbolic variables. Also this formula was mentioned in article [5] (25). I tried use formula, mentioned in lectures, but it didn't gave appropriate results, and did not satisfy the expression $w = (KK^T)^{-1}$ and conflicted with logic of parameters $a = f_x/f_y$:

$$w_{lecture\ slides} = \begin{bmatrix} a^2 & 0 & -u_0 a^2 \\ 0 & 1 & -v_0 \\ -u_0 a^2 & -v_0 & f_y^2 + a^2 u_0^2 + v_0^2 \end{bmatrix}, \text{ where } a = f_x/f_y$$

Therefore I didn't use this formula from lecture slides.

To define the required four constraints I used calibration from rectified face plus orthogonal vanishing point. It include:

- The reconstructing homography Hr from given img to rectified img
- The image of the line at the infinity l'_∞
- The vanishing point v along the direction orthogonal to the face (in our terms face is horizontal plane)

From plane-to-image homography $H = [h_1, h_2, h_3]$, where $H = H_R^{-1}$

$\rightarrow$

$$h_1^T w h_2 = 0 \text{ (constraint 1)}$$

$$h_1^T w h_1 - h_2^T w h_2 = 0 \text{ (constraint 2)}$$

$$\text{from } v_1^T w v = 0 \text{ and } v_2^T w v = 0, \text{ and } l'_\infty = v_1 \times v_2 \rightarrow$$

$$l'_\infty = w v \rightarrow$$

$$v^T w h_1 = 0, \text{ and } v^T w h_2 = 0 \text{ (constraint 3 and 4)}$$

v – vanishing point along the direction orthogonal to the face

Using these four constraints I found all parameters, describing the calibration matrix. As a vanishing point I used the intersection of lines AE and DF . To verify found results I computed constraints using w and received zeroes, the same operation I made for the relationship between w and K and also received zeroes. I also have tried to use Cholesky factorization to find K from w [6], but it did not provide appropriate results and k_{12} in matrix K was not equal to zero.

$$K = \begin{bmatrix} 708.4832 & 0 & 813.9804 \\ 0 & 766.1626 & 563.7740 \\ 0 & 0 & 1.0000 \end{bmatrix}$$

Part 4. Height of parallelepiped

To find the height of parallelepiped I used the same technique, as in part 2 with direct method, using SVD decomposition. By definition, circular points are the intersections of the circumference and the line at infinity. Line at infinity for the vertical plane was defined as the intersection of AD with EF and AE with DF , result presented in Figure 10.

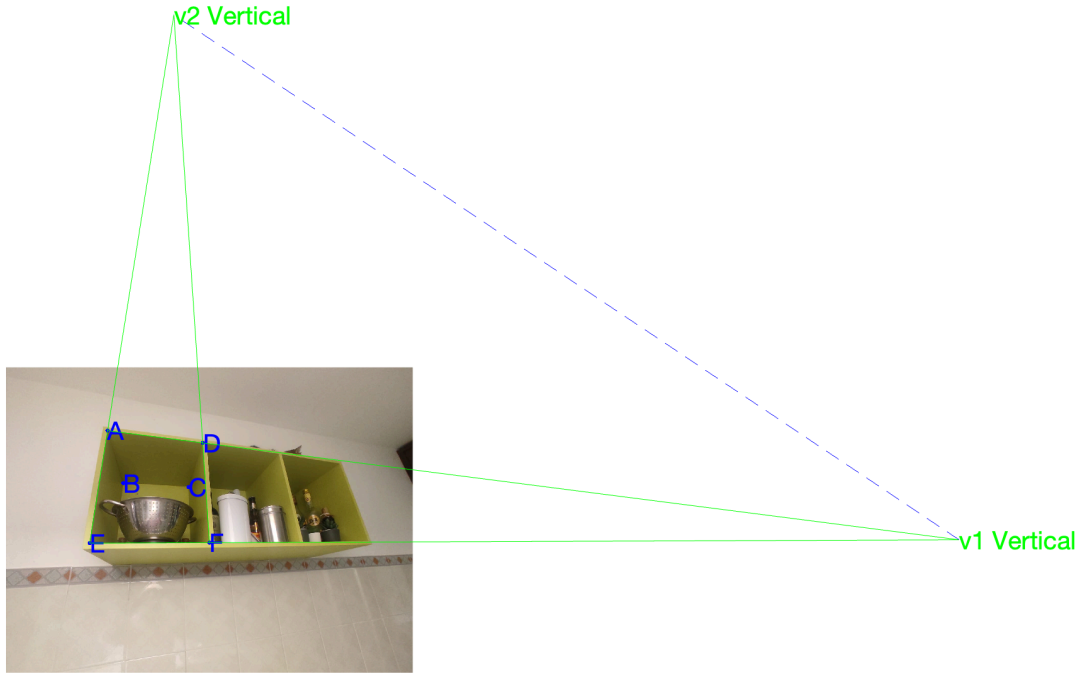


Figure 10 - Line at infinity for vertical plane.

As a circumference I used IAC from the previous part. Founded rectification matrix was equal to:

$$H_{vert\ rect} = \begin{bmatrix} -0.9999 & -0.0152 & -0.0004 \\ 0.0266 & -1.7434 & 0.0010 \\ -0.0004 & 0.0005 & 1.0000 \end{bmatrix}$$

Result of rectification presented in figure 11.



Figure 11 - Result of rectification of vertical plane.

Given the total length of the parallelepiped and the information that AD is one-third of that length, we can determine the height of parallelepiped (DF). By using this proportion, the calculated height of DF is **0.36288**.

Part 5. Curve S

To determine the coordinates of curve S , I performed a rectification of the plane where this curve lies. This involved identifying the midpoints of the lines DF and HG (on the front plane) and their corresponding analogues on the back plane. The steps are detailed below:

1. Finding the Midpoints of DF and HG :

- Using the rectification process described in **Part 4**, I first rectified the vertical plane to locate the midpoints of DF (point K) and HG (point P).
- After identifying these midpoints in the rectified plane, I applied the reverse transformation to determine their positions in the source image.

2. Finding the Midpoints of the Back Vertical Edges:

- To locate the midpoints of the back vertical edges, I used the rectification process from **Part 2**.
- First, I identified a reference point R on the back edge, represented by a line passing through C and R .
- Using the rectification of the horizontal plane, I found the midpoint of the back edge as the intersection of:
 - The line CR , and
 - A line passing through K (the midpoint of DF) that is parallel to CD .
- The resulting point is R_int , which represents the midpoint of the left back edge.
- Similarly, I applied the same procedure to the right back edge (line MN), finding the midpoint N_int as the intersection of:
 - The line MN , and
 - A line passing through P (the midpoint of HG) that is parallel to HG .

3. Defining the Vanishing Line:

- With the midpoints R_int and N_int of the back edges identified, I computed the vanishing line corresponding to this plane.
- While I could have used the vanishing line derived in **Part 2**, the positions of R_int and N_int are necessary for accurate computation of the ellipse's position on this plane. Without these points, the ellipse would be incorrectly scaled.

The identified midpoints and the vanishing line are illustrated in **Figure 12**.

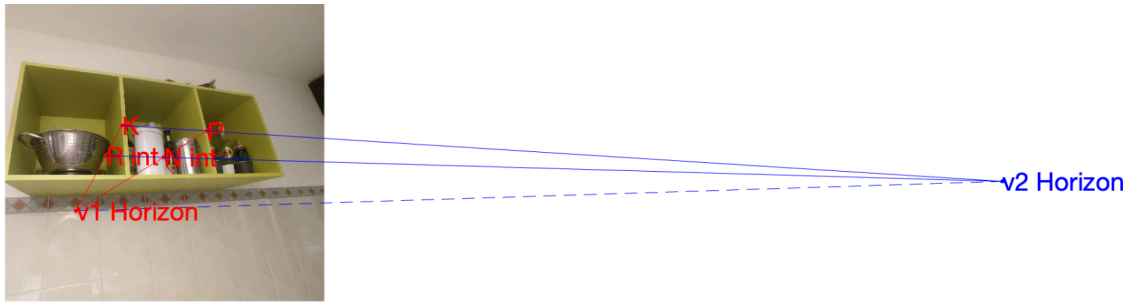


Figure 12 - Vanishing line of horizontal plane, based on middle points.

Knowing the vanishing line and calibration matrix, I made a rectification of the plane where the curve S lies, as in part 4. Result of rectification of plane, containing S , presented in figure 13.



Figure 13 - Rectification of plane, containing curve S .

I also applied rectification to points on the plane and to curve S . Their coordinates correspond to point K presented in Figure 14. Points are flipped vertically and horizontally, but this should not prevent us from finding the coordinates of the curve S relative to K .

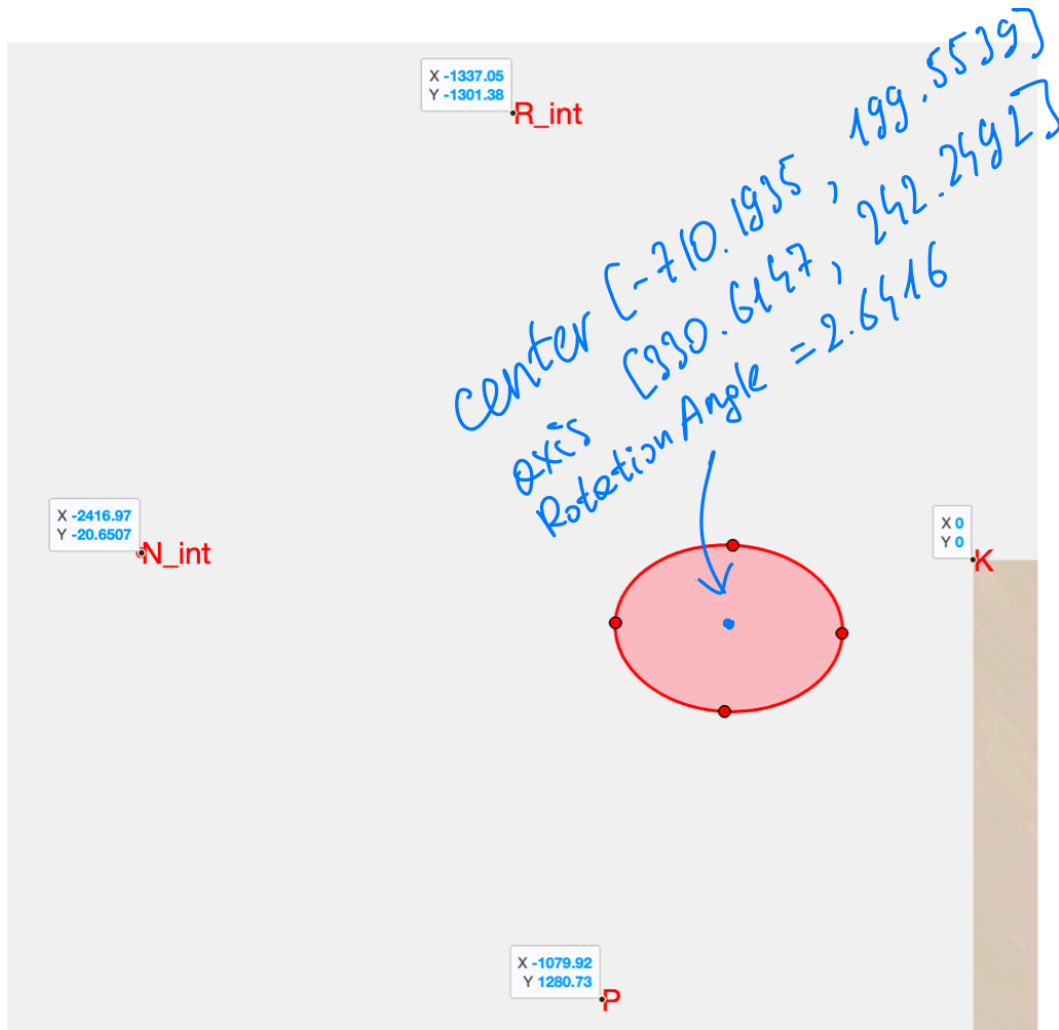


Figure 14 - Rectification of plane with curve S and point on same plane.

To determine the real-world coordinates of the ellipse center, its axis lengths, and rotation angle, I used the following approach:

1. **Ellipse Center Projection:**
 - I projected the ellipse center onto the lines KP and KR_{int} .
 - Using these projections, I calculated the appropriate scale factors to accurately position the ellipse within the plane.
2. **Final Coordinates and Adjustments:**
 - Taking into account that the ellipse center is located at half the height of the parallelepiped, I determined the real-world coordinates of the ellipse center relative to point E .
 - The computed center coordinates are: **Center: [0.4548, 0.1814, -0.0673]**.
3. **Ellipse Properties:**
 - The axis lengths of the ellipse were determined to be: **Axis Lengths: [0.0482, 0.0658]**.
 - The rotation angle of the ellipse was calculated as: **Rotation Angle: -2.6416 radians** (minus, because take into account flipping).

Part 6. Localisation

To locate the camera I had to define a projection matrix from 3D space to 2D image. Formulas (saving notations of report) and ideas to solve this part were taken from article [7].

$$\text{Generally : } \bar{x} = H_{world2img} \bar{X}$$

$$H_{world2img} = \lambda K [R \ t], \text{ where } R - \text{rotation matrix, } t - \text{translation vector} \\ \lambda - \text{arbitrary scalar}$$

$$R = [r_1 \ r_2 \ r_3], H_{world2img} = [h_1 \ h_2 \ h_3]$$

assuming all of that we can define r_1, r_2, r_3 and $t \implies$

$$r_1 = \lambda A^{-1} h_1, r_2 = \lambda A^{-1} h_2, r_3 = r_1 \times r_2, t = \lambda A^{-1} h_3$$

$$\lambda = 1 / \|A^{-1} h_1\| = 1 / \|A^{-1} h_2\|$$

Because of noise in data the so-computed matrix R does not in general satisfy the properties of a rotation matrix. To solve this problem I had to use SVD decomposition and update the received matrix R . This trick also was mentioned in [7] (Approximating a 3×3 matrix by a Rotation Matrix):

$$SVD(R) = USV^T$$

$$R_{rotation} = UV^T$$

In our task I firstly had to rectify vertical plane and after that define projectivity from world to image:

$$\begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = H_{img2world} \cdot H_{vert} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \text{ where } X, Y - \text{real world coordinates,}$$

x, y – image points coordinates, without loss of generality, we assume the $Z = 0$

$$\begin{bmatrix} x_\pi \\ y_\pi \\ 1 \end{bmatrix} = H_{vert} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \text{ where } H_{vert} - \text{rectification matrix from part 4}$$

x_π, y_π – image points after rectification

$$H_{world2img} = (H_{img2world} \cdot H_{vert})^{-1}$$

Camera position can be found, assuming that after projection its position is equal to 0, since the camera is located at the origin of its own frame.

$$C_{pos} = -R^{-T}t$$

since R – rotation matrix, $R^{-1} = R^T$

Received coordinates of camera, corresponding to parallelepiped is equal to **[0.2046, -0.1268, 0.5472]**. Camera position corresponding to the zero coordinates in point E, presented in figure 15.

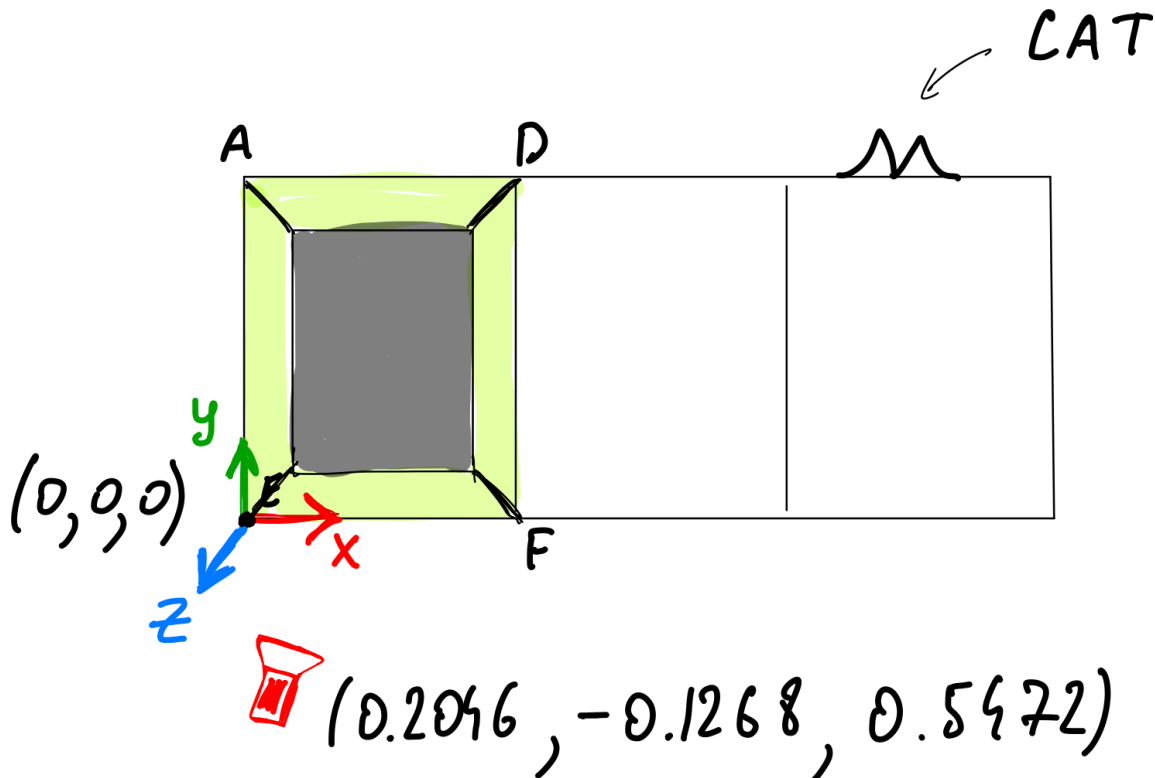


Figure 15 - Camera position corresponding to point E

Different views of 3D recovered models presented in figure 16-19, where center is point E, Y axis - green, X axis - red, Z axis - blue.

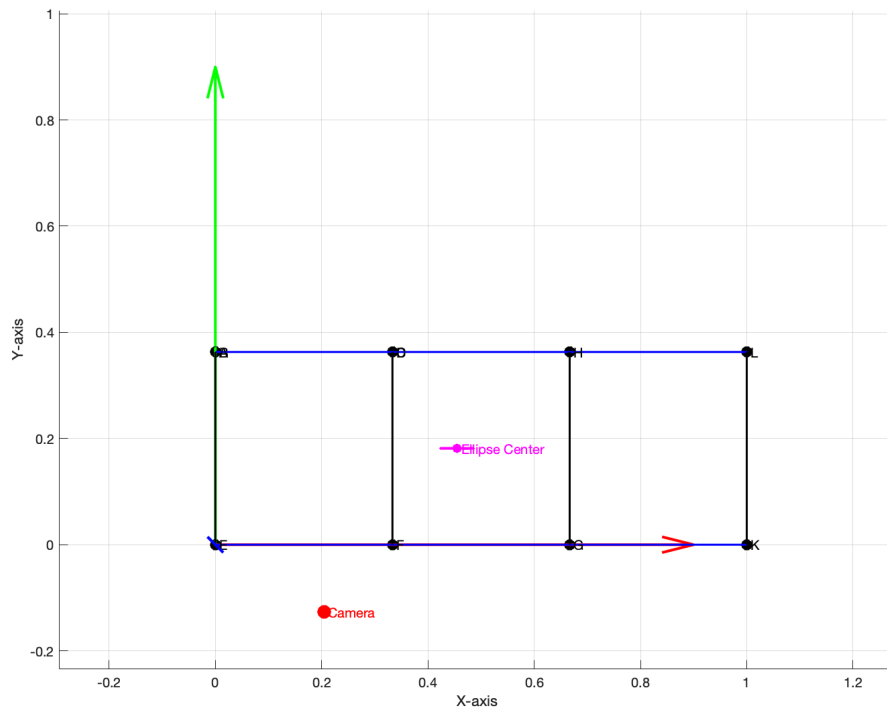


Figure 16 - YX plane of scene

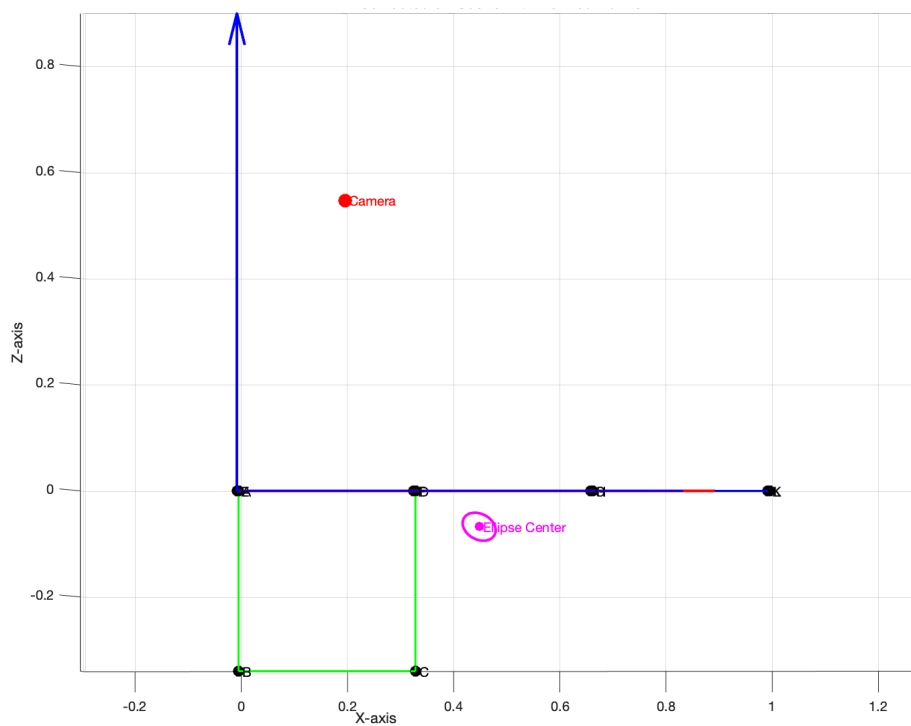
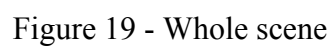
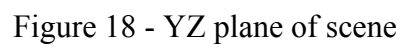


Figure 17 - ZX plane of scene



In matlab, you can view the final 3D model for yourself.

References

1. Canny Edge detection <https://it.mathworks.com/help/images/ref/edge.html>
 2. Hough Transform <https://it.mathworks.com/help/images/hough-transform.html>
 3. Harris Features <https://it.mathworks.com/help/vision/ref/detectharrisfeatures.html>
 4. Luca Magri IACV materials <https://magrilu.github.io/iacv/>
 5. Calibration Method Based on the Image of the Absolute Quadratic Curve. Wenlei Liu; Sentang Wu; Xiaolong Wu; Hongbo Zhao.
<https://ieeexplore.ieee.org/document/8616767>
 6. Cholesky factorization <https://it.mathworks.com/help/matlab/ref/chol.html>
 7. A Flexible New Technique for Camera Calibration. Zhengyou Zhang.
<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr98-71.pdf>
-

Folder structure:

- **HW**
 - **ellipse_functions** (folder with functions to work with ellipses)
 - **feature_extracture_functions** (folder with functions to extract features)
 - **images** (folder with source img and all images received during the solution of the tasks)
 - **part1.m - part6.m** - files solving corresponding task from assignment
 - **part0.m** - file to extract features and define corners
 - **part7.m** file to plot final 3D scene with curve S